

XCPC 杂题选讲

dXqwq

2026-05-21

Who am I?

不重要。

About XCPC

XCPC 是一类三人一机五小时签到题比赛。

题目均来自我们队伍参赛/验题的场次。

- 我们过了的题：武汉 A，南京 A，郑州 E，UcupF A
- 我们会了但没过的题：成都 H，CCPCF D
- 我们不会的题：成都 F，CCPCF C

武汉 A: Easy

给定 k_1, k_2, b_1, b_2, p, n 。

求 $x = 0, 1, \dots, n$ 中有多少 x 满足 $(k_1x + b_1) \bmod p \leq (k_2x + b_2) \bmod p$ 。

$T \leq 10^4, 0 \leq k_1, k_2, b_1, b_2, n < p \leq 10^6$

武汉 A

Hint 1: 此题并不难。

武汉 A

Hint 1: 此题并不难。

Hint 2: 这个题可以做 $0 \leq k_1, k_2, b_1, b_2, n < p \leq 10^9$

Hint 1: 此题并不难。

Hint 2: 这个题可以做 $0 \leq k_1, k_2, b_1, b_2, n < p \leq 10^9$

$$[x \bmod p + y \bmod p \geq p] = \left\lfloor \frac{x+y}{p} \right\rfloor - \left\lfloor \frac{x}{p} \right\rfloor - \left\lfloor \frac{y}{p} \right\rfloor$$

将 k_2, b_2 取负号后使用类欧几里得算法即可, $O(T \log p)$ 。

成都 F: Hard

构造题。

构造一个 $n \times m$ 的 LURD 网格，使得从 $(1, 1)$ 开始恰好走 k 步可以到达 (n, m) 。

每一步会照着格子方向前进一步，但如果会走出网格就不前进。走完之后，原来格子的方向会反转（上下互换，左右互换）。

给定 k ，你可以自己指定 n, m ，但需要保证 $1 \leq n, m \leq 8$ 。

$k \leq 10^6$ 。

成都 F

Hint: 先构造一个网格, 使得答案尽可能大。

Hint: 先构造一个网格, 使得答案尽可能大。

不难发现我们需要递归的结构, 但如何制造递归的结构?

n 个点的状态数是 $O(2^n)$ 级别的, 考虑一般转移链上的情况:

Hint: 先构造一个网格, 使得答案尽可能大。

不难发现我们需要递归的结构, 但如何制造递归的结构?

n 个点的状态数是 $O(2^n)$ 级别的, 考虑一般转移链上的情况:

- 一维的情况答案只能构造到 $O(n^2)$ 量级, 对应每个点往前一个点转移。
- 而如果我们每个点往第一个点转移, 答案就是 $O(2^n)$ 量级的了。

Idea: 如果我们构造一条路径, 方向错误时向尽早的结点转移, 答案就会更大!

成都 F

```
"rrrrrrrd",  
"dlllllll",  
"drrrrrrd",  
"dudlllll",  
"dudrrrrd",  
"dududlll",  
"dududrrd",  
"rurururr",
```

最后反转所有格子即可。

进一步可以发现将每个格子置为正向和反向对结果的影响是独立的（都只影响第一次经过该格子时是否要回溯一遍前若干个格子的过程），62 个位置的背包经过验证可以覆盖 $64 \sim 10^6$ 的所有情况。

最后需要特判 k 较小的构造。

成都 H: Medium

给定 n 个物品，每个物品有重量 w 和价值 v ，其中的至多一个参数是缺失的。

补全这些参数使得以下两个算法在容量为 W 的背包问题上得到相同的物品集合：

- 先将所有物品根据重量优先排序，然后从前到后贪心拿取。
- 先将所有物品根据价值优先排序，然后从前到后贪心拿取。

注意上方排序的第二关键字是物品的输入顺序。

构造一组方案，或者判定无解。

$$n \leq 3000, 1 \leq w_i, r_i \leq 10^9, 1 \leq V \leq 10^9$$

Hint: 注意重量和价值并不是对称的参数！从哪边入手更简单？

Hint: 注意重量和价值并不是对称的参数！从哪边入手更简单？

从重量入手更简单，因为取走的是一个极长前缀。

假设所有的 w_i 都是已知的，我们只需要将对应要取的 v_i 设为上限 10^9 ，不取的 v_i 设为下限 1 即可。

所以枚举我们从已知的 w_i 中取走了前几小，此时所有 v_i 均已知。

现在再从价值侧考虑（因为此时所有物品的价值都是已知的），我们尝试归纳子问题：

现在的问题是，有若干物品，重量已知的我们知道必须选/必须不选。

现在的问题是，有若干物品，重量已知的我们知道必须选/必须不选。

不妨设重量未知的都得选上（选不上的视为重量为 0），我们每次只需要求出一个极大的重量满足该被选上的能选上即可。这里极大的重量受到的主要限制减去这个重量之后不能低于该被选上的重量之和，也不能高于任何不该被选上的重量。

结束了.....?

现在的问题是，有若干物品，重量已知的我们知道必须选/必须不选。

不妨设重量未知的都得选上（选不上的视为重量为 0），我们每次只需要求出一个极大的重量满足该被选上的能选上即可。这里极大的重量受到的主要限制减去这个重量之后不能低于该被选上的重量之和，也不能高于任何不该被选上的重量。

结束了.....?

注意即使我们将选不上的都设为 10^9 重量，我们也可能选到其中一个！

题解讲了一种很简单的规避情况，事实上也可以通过真正进行很多分讨处理该情况，但是为了让大家不要被我的绕口令弄晕，我们留给大家自己处理。

交互题！

有一个 $n \times m$ 的网格，中间有一个洞，剩下的位置有一个人。人走到洞里和格子外面都会死。

你可以下 $(n + m + 10)$ 次指令，每次指令可以让所有活人都向上/下/左/右走一步。

每次指令后你会知道存活的人数，你需要求出洞在哪里。

$$3 \leq n, m \leq 30$$

假设洞在 (x, y) 。

- Hint: 不妨先假设 (x, y) 不在边界上, 如何求出洞在哪里?

假设洞在 (x, y) 。

- Hint: 不妨先假设 (x, y) 不在边界上, 如何求出洞在哪里?

首先, 一直让所有人往下走。

对于没有洞的列, 显然每次会死一个人。

对于有洞的列, 前 $\min(x - 1, n - x)$ 轮会死两个人, 后若干轮会死一个人。

综上, 可以通过差分求出每轮死了几个人, 就可以求出洞的两个可能行, 且两种情况对称。

南京 A

需要区分这两种情况,

0000	0000
0x00	0000
1011	1111
1011	1x11
1111	1011

南京 A

需要区分这两种情况， 走一次左/右就可以。

0000	0000
0x00	0000
1011	1111
1011	1x11
1111	1011

南京 A

需要区分这两种情况， 走一次左/右就可以。

0000	0000
0x00	0000
1011	1111
1011	1x11
1111	1011

然后可以将其推到底部， 来保证仅有洞的所在列空缺。进一步向某个方向移动即可。

```
000000
000x00
011010
```

注意这里角落缺失是因为走左/右造成的信息损失。

考虑上述过程并不依赖行是否在边界，对列是否在边界的依赖主要来自底部信息。

考虑上述过程并不依赖行是否在边界，对列是否在边界的依赖主要来自底部信息。

可以将底部信息加宽！我们可以将一个完整的行（可以证明存在这样的一行）在洞口再走一次左/右来制造宽度为 2 的信息，这样就可以规避边缘损失。

```
..1110 => Row 1  
...110 => Row 2  
0...10 => Row 3  
01...0 => Row 4  
011... => Row 5  
0111.. => Row 6
```

但这种辨认方法基于宽度 ≥ 4 , 如何解决?

.10 => Row 1

..0 => Row 2

0.. => Row 3

(这个不行)

.110 => Row 1

..10 => Row 2

0..0 => Row 3

01.. => Row 4

(这个可以)

我们可以转置矩阵，所以 $\max(n, m) \geq 4$ 即可使用上文的方法。

剩下的情况只有 3×3 ，手搜决策树即可通过。

步数为 $n + m + O(1)$ 。

郑州 E: Easy

有 n 个同心圆，每个圆上有一些缝。

给定 q 次询问，每次给出某个圆上的一个位置，求出从圆心出发到该点的最短路。

$$n, m, q \leq 2 \times 10^5, r_i \leq 10^8$$

郑州 E

Hint 1: 暴力怎么做?

Hint 1: 暴力怎么做?

相当于在每一层选择一个缝，缝之间需要沿着圆弧走一段，然后再走一段直线。算出距离的部分是相对简单的，进行 DP 可以做到 $O(m^2)$ 。

Hint 1: 暴力怎么做?

相当于在每一层选择一个缝, 缝之间需要沿着圆弧走一段, 然后再走一段直线。算出距离的部分是相对简单的, 进行 DP 可以做到 $O(m^2)$ 。

Hint 2: 最短路树会长什么样?

Hint 1: 暴力怎么做?

相当于在每一层选择一个缝, 缝之间需要沿着圆弧走一段, 然后再走一段直线。算出距离的部分是相对简单的, 进行 DP 可以做到 $O(m^2)$ 。

Hint 2: 最短路树会长什么样?

交叉是不优的 (字面意义上)!

因此需要做一个环上的四边形不等式优化 DP。

Hint 1: 暴力怎么做?

相当于在每一层选择一个缝, 缝之间需要沿着圆弧走一段, 然后再走一段直线。算出距离的部分是相对简单的, 进行 DP 可以做到 $O(m^2)$ 。

Hint 2: 最短路树会长什么样?

交叉是不优的 (字面意义上)!

因此需要做一个环上的四边形不等式优化 DP。

和链上不一样的是, 环上距离存在方向.....

经过一些带方向距离的精细实现后可以在 $O(m \log m)$ 的时间复杂度内求解。

杭州 L: Medium

构造一个如下对象之间的双射:

- 根为 1, $fa_i < i$, 叶子集合为 S 的树。
- 根为 n , $fa_i > i$, 叶子集合为 $[n] \setminus S$ 的树。

形式化地, 你需要构造两个函数:

- $f(x)$ 为给定第一类对象, 输出第二类对象的函数。
- $g(x)$ 为给定第二类对象, 输出第一类对象的函数。
- $f(g(x)) = x$ 。

每组数据会先给你 $T \leq 10^5$ 棵大小为 $n \leq 10^3$ 的树作为第一次的输入, 且 $\sum n^2 \leq 10^7$, 你的程序在第一次的输出会作为第二次的输入。

Fun fact: 我们是怎么做不出来的?

我想了若干个小时的数量关系, 发现和斯特林数有关。

但问题是, 这样的自然顺序是反的, 也就是 $[n] \setminus S$ 的 $x \rightarrow n + 1 - x$ 。

要修需要将一个阶乘进制的整数反转, 最后遗憾没有写完。

归纳构造。假设我们已经对 $n - 1$ 的树构造了一个合法的双射，对第 n 个点的连边，该如何处理？

归纳构造。假设我们已经对 $n - 1$ 的树构造了一个合法的双射，对第 n 个点的连边，该如何处理？

在第一棵树上，

- 新边可能干掉了一个叶子，也可能没有干掉叶子。
- 新点一定是叶子。

在第二棵树上，

- 第一棵树上干掉的点 x 一定要变成叶子。
- 新点 y 一定不是叶子。
- 剩下的点情况必须不变。

杭州 L

简单的想法：把 x 的孩子全扔到 y 上，但这不是一个双射。

(其实这里因为我们需要写逆变换代码，也可以近似认为难以求逆)

还没用到的性质是扔出去的孩子编号都比 x 要小。

杭州 I

简单的想法：把 x 的孩子全扔到 y 上，但这不是一个双射。

(其实这里因为我们需要写逆变换代码，也可以近似认为难以求逆)

还没用到的性质是扔出去的孩子编号都比 x 要小。

稍微复杂的想法：把 x 的孩子丢到 fa_x 上，把 fa_x 的孩子丢到 fa_{fa_x} 上，以此类推。

此时性质保证了.....

杭州 L

简单的想法：把 x 的孩子全扔到 y 上，但这不是一个双射。

(其实这里因为我们需要写逆变换代码，也可以近似认为难以求逆)

还没用到的性质是扔出去的孩子编号都比 x 要小。

稍微复杂的想法：把 x 的孩子丢到 fa_x 上，把 fa_x 的孩子丢到 fa_{fa_x} 上，以此类推。

此时性质保证了.....

如果一个点的孩子是从孙子升级而来的，那么它没有经过升级的儿子一定是标号最大的点。(因为之前这些孙子都是这个儿子的儿子)

因而可以从 n 开始走标号最大的点来进行逆操作。

时间复杂度 $O(\sum n^2)$ 。

CCPC Final C: Medium

给定 p, q 和四个物品 (a_i, b_i) , 你需要求出同余背包的最小解。

具体地, 你可以以 b_i 每个的代价拿取体积为 a_i 的物品, 需要最小化体积和 $\bmod p = q$ 的代价。

- $p \in \mathbb{P}, \sum p^{0.5} \leq 2^{17}, 1 \leq q, a_i < p, 1 \leq b \leq 10^9$

3 s, 1024 MB

$$a_1x_1 + a_2x_2 + a_3x_3 + a_4x_4 \equiv q \pmod{p}$$

观察: $\prod(x_i + 1) \leq p$ 。

考虑反证法, 则可以找到两组 (y_1, y_2, y_3, y_4) 满足 $a_1y_1 + a_2y_2 + a_3y_3 + a_4y_4 \equiv 0$ 。

$$a_1x_1 + a_2x_2 + a_3x_3 + a_4x_4 \equiv q \pmod{p}$$

观察: $\prod(x_i + 1) \leq p$ 。

考虑反证法, 则可以找到两组 (y_1, y_2, y_3, y_4) 满足 $a_1y_1 + a_2y_2 + a_3y_3 + a_4y_4 \equiv 0$ 。

将它们作差并记 $x^1 = x - \Delta y, x^2 = x + \Delta y$ 。

注意到 $\Delta y_i \leq x_i$, 所以一定不会退位, 不考虑进位则 x^1, x^2 的代价和是 x 的代价的两倍 (考虑进位则可能更小), 因此有两种情况:

- 要么存在一个 x^t 代价更小, 这是矛盾的。
- 要么存在一个 x^t 字典序更小, 但字典序是全序不可能一直迭代, 这也是矛盾的。

因此我们可以枚举 x_i 的相对大小关系，不妨设 $x_1 \leq x_2 \leq x_3 \leq x_4$ 。

根据上述原理进行枚举，可以证明方案数不超过.....

因此我们可以枚举 x_i 的相对大小关系，不妨设 $x_1 \leq x_2 \leq x_3 \leq x_4$ 。

根据上述原理进行枚举，可以证明方案数不超过..... $p^{\frac{3}{4}}$!

加上大量剪枝是可以 $O(p^{\frac{3}{4}})$ 通过的，这也是 Universal Cup 唯一的通过的做法。

因此我们可以枚举 x_i 的相对大小关系，不妨设 $x_1 \leq x_2 \leq x_3 \leq x_4$ 。

根据上述原理进行枚举，可以证明方案数不超过..... $p^{\frac{3}{4}}$!

加上大量剪枝是可以 $O(p^{\frac{3}{4}})$ 通过的，这也是 Universal Cup 唯一的通过的做法。

另一个做法是枚举 (x_1, x_2) 和 x_3 ，各有 $p^{\frac{1}{2}}$ 种方案。

但注意到此时 a_4 已经固定，可以将所有数都乘上 a_4^{-1} 。

因此我们可以枚举 x_i 的相对大小关系，不妨设 $x_1 \leq x_2 \leq x_3 \leq x_4$ 。

根据上述原理进行枚举，可以证明方案数不超过..... $p^{\frac{3}{4}}$!

加上大量剪枝是可以 $O(p^{\frac{3}{4}})$ 通过的，这也是 Universal Cup 唯一的通过的做法。

另一个做法是枚举 (x_1, x_2) 和 x_3 ，各有 $p^{\frac{1}{2}}$ 种方案。

但注意到此时 a_4 已经固定，可以将所有数都乘上 a_4^{-1} 。

然后变成了一维问题，可以排序之后做单调队列， $O(p^{\frac{1}{2}} \log p)$ 。

CCPC Final D: Easy

给定一个字符串 S 和 n 个操作 a_i 。

- 操作 x : 将 $S \leftarrow \text{prefix}(S^\infty, x)$ 。

你需要重排 a_i 的顺序，最大化依次执行后得到的字符串的字典序，输出方案。

$$n, |S| \leq 10^5, a_i \leq 10^9$$

- Hint 1: 什么样的操作是有用/没用的?

- Hint 1: 什么样的操作是有用/没用的?

在极小的 a_i 之前的操作没有用。

进一步地, 只有 a_i 的后缀最小值有用。

显然我们可以构造所有后缀最小值集合的排列, 因此可以转化问题。

- 先将 a_i 排序, 对于非最小值依次决定是否执行这个操作。

显然所有字符串可以根据 $S^\infty < T^\infty$ 建立全序关系。

现在的问题是快速判断 $[S^\infty < T^\infty] \Leftrightarrow [S + T < T + S]$ 。

显然所有字符串可以根据 $S^\infty < T^\infty$ 建立全序关系。

现在的问题是快速判断 $[S^\infty < T^\infty] \Leftrightarrow [S + T < T + S]$ 。

- Hint 2: 可以二分+哈希找到第一个不同的位置。

显然所有字符串可以根据 $S^\infty < T^\infty$ 建立全序关系。

现在的问题是快速判断 $[S^\infty < T^\infty] \Leftrightarrow [S + T < T + S]$ 。

- Hint 2: 可以二分+哈希找到第一个不同的位置。

二分要计算前缀哈希值，如何处理？

计算每轮迭代的哈希值（多次重复的串可以快速幂做），向前递归。

注意到每次递归的长度是取模的，因此只需要迭代 $\log a$ 轮。

UCup Final A: Medium

给定一个 01 串 S 。

你每次可以选择一个 1，删除距离其 $\leq k$ 的所有字符并合并剩余字符。

问是否能删空整个串，如果能，需要再求出最小次数。

$$1 \leq |S|, k \leq 10^6$$

Hint 1: 先考虑什么样的 Pattern 能删空。

Hint 1: 先考虑什么样的 Pattern 能删空。

不难想到将所有位置分为必须选择和不选择两类。

相当于 01 串中，每次只能删除 $0\dots 010\dots 0$ 的子串。

进一步地，我们将两边也填补适量的 0 以忽略边界情况。

现在的合法条件是.....

UCup Final A

Hint 2: 猜猜结论，打个表看看还差多少。

Hint 2: 猜猜结论，打个表看看还差多少。

显然的必要条件：任意两个 1 之间的距离至少是 $k + 1$ ，1 的数量一定是 $\frac{n}{2k+1}$ 。

一个不合法的情况： $n = 15, k = 2$ 时 000100001001000 是做不到的。

可以用间距 0 的个数描述一个序列，例如

00**1**00000**1**000**1**00 $\Rightarrow [2, 5, 3, 2]$

每次合并相当于选择两个 $\geq k$ 的数加起来 $-2k$

00**1**000**001**00**01**00 $\Rightarrow [2, 5, 3, 2]$

00**1**000_____0**1**00 $\Rightarrow [2, 4, 2]$

000**1**0000**1**00**1**000 $\Rightarrow [3, 4, 2, 3]$ 可以验证做不到。

排除简单的条件 $\sum a_i = (|a| - 1)2k, a_i \geq k$ 之后仍有不合法的例子。

- Hint 3: 考虑最后一次合并。

排除简单的条件 $\sum a_i = (|a| - 1)2k, a_i \geq k$ 之后仍有不合法的例子。

- Hint 3: 考虑最后一次合并。

最后一次合并一定是 $[k, k] \Rightarrow 0$, 所以相当于要将两边都变成 k 。

变成 k 是比变成 0 更好判断的条件, 为什么?

排除简单的条件 $\sum a_i = (|a| - 1)2k, a_i \geq k$ 之后仍有不合法的例子。

- Hint 3: 考虑最后一次合并。

最后一次合并一定是 $[k, k] \Rightarrow 0$, 所以相当于要将两边都变成 k 。

变成 k 是比变成 0 更好判断的条件, 为什么?

此时前述条件充要。具体地, 将每个数 $-k$, 贪心合并正数不破坏性质, 然后由于 $\sum (a_i - k) = -k$, 贪心合并所有负数也不会不合法。

考虑如何将上述结论代回原题？

考虑如何将上述结论代回原题？

序列判定：相当于只需要枚举最后合并的位置，检查两边是否满足该条件即可。

原题：枚举最后合并的位置后，如何取 1 的子集？

考虑如何将上述结论代回原题？

序列判定：相当于只需要枚举最后合并的位置，检查两边是否满足该条件即可。

原题：枚举最后合并的位置后，如何取 1 的子集？

从中心向两边贪心取最小的间距即可。如果选多了，总是可以通过直接扔掉几个来实现更少的效果的。

这里有一些实现细节，但整体代码很小清新。

时间复杂度 $O(n)$ 。